SeedHammer II — the hashlock vault

Six seeds, three engraved passphrases, four degrading tiers, one taproot tree.

A wallet that hands control on in stages. Three keys and a passphrase can spend today; two other keys and a second passphrase can spend after a relative delay; a sixth key alone can spend after an absolute height; and after a longer height **a passphrase alone** can spend — no key at all.

```
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc1249c79aa840de94fa4a85bf55a453c813),multi_a(3,@0/270028'/0'/0'/0'/<0;1>/*,@1/270028'/0'/0'/0'/<0;1>/*,@2/270028'/0'/0'/0'/<0;1>/*)),{and_v(v:older(32768),and_v(v:sha256(9ce09a8df1d1f8bc8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,@3/270028'/0'/0'/0'/<0;1>/*,@4/270028'/0'/0'/0'/<0;1>/*))),{and_v(v:after(1173520),pk(@5/270028'/0'/0'/0'/<0;1>/*)),and_v(v:after(1383520),sha256(4743d7c47df21d29e3ed3dfec5d0c0a884ccc2708637dddf771c36d214056954))}}})
```

tier	who can spend	when
1	3-of-3 (@0,@1,@2) + passphrase A	any time
2	2-of-2 (@3,@4) + passphrase B	`older(32768)` — relative, ~7.6 months after funding
3	@5 alone	`after(1173520)` — 963,520 + 210,000
4	**passphrase C alone**	`after(1383520)` — 963,520 + 420,000

Test material only — never put funds behind these keys.
The six seeds come from entropy `0x0…01` through `0x0…06` and the three passphrases are printable strings in `derive-hashvault-keys.sh`. Everything here is regenerable by design; that is the point, and it is also why it is worthless.

The internal key is the BIP-341 NUMS point
, so there is no key-path spend and every spend reveals which tier was used.

The source material

One seed per key, so every key has its own master fingerprint. That is the difference from this constellation's other multi-key journey, where three masters supplied eleven keys across four accounts.

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/seeds/key-0.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon aban
don abandon abandon abandon abandon abandon abandon abandon diesel
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/seeds/key-1.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon aban
don abandon abandon abandon abandon abandon abandon abandon false
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/seeds/key-2.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon aban
don abandon abandon abandon abandon abandon abandon abandon kite
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/seeds/key-3.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon aban
don abandon abandon abandon abandon abandon abandon abandon organ
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/seeds/key-4.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon aban
don abandon abandon abandon abandon abandon abandon abandon ready
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/seeds/key-5.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon aban
don abandon abandon abandon abandon abandon abandon abandon surface
[exit 0]
```

Derived at the constellation's own path

`m/270028'/coin'/account'/script'` — purpose `270028'` reads bg002h in a single component, and level 4 is the script type with `0'` = tr. Ruled 2026-08-18.

> **The ruling had no implementation until this journey needed it.**
>
> `ms derive` offered only the BIP templates and `mk derive --path` is relative-and-unhardened-only, so nothing in the constellation could derive at its own path. `ms derive --template bg002h-tr` is what closed that.

```
$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms derive --phrase abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon aban
don abandon diesel --template bg002h-tr --account 0
warning: secret material on argv (--phrase) — pipe via --phrase - to avoid /proc/$PID/cmdline exposure
master_fingerprint:  39ec1b6e
account_path:        m/270028'/0'/0'/0'
account_xpub:        xpub6E6MpT6SMBy1Poaj5rEJ4DN5E73e58TWta6SVLs4eYpJ6hDNKqhwLxFcvMUtJnDSQJpdkThXEma9t23WiHhrYAJbK9GUu672njH
HFenKt88
language:            english (DEFAULT)
note: --language defaulted to english; the master fingerprint and xpub depend on the wordlist language (record it alongside
the backup)
note: stdout is watch-only — public keys only, cannot spend
[exit 0]

The six key files this produced:

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/keys/key-0.xpub
# @0 — tier 1 — 3-of-3 with hash A, spendable at any time
# seed entropy 0x00000000000000000000000000000000000000000000000000000000000000000000000000000001 (TEST ONLY), origin [39ec1b6e/270028'/0'/
0'/0']
xpub6E6MpT6SMBy1Poaj5rEJ4DN5E73e58TWta6SVLs4eYpJ6hDNKqhwLxFcvMUtJnDSQJpdkThXEma9t23WiHhrYAJbK9GUu672njHHFenKt88
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/keys/key-1.xpub
# @1 — tier 1 — 3-of-3 with hash A, spendable at any time
# seed entropy 0x00000000000000000000000000000000000000000000000000000000000000000000000000000002 (TEST ONLY), origin [d02bcedf/270028'/0'/
0'/0']
xpub6F5JvRbobuuiwNQLy9ifRCK1HbewqyYk3PkMPNsrgkPcGGuDYXdYUDmHqMsQeBzq58tf8eRxLaTWJPaa1pcE9TmRZTAFVk7U61UxQtMUNFj
[exit 0]
```

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/keys/key-2.xpub
# @2 — tier 1 — 3-of-3 with hash A, spendable at any time
# seed entropy 0x0000000000000000000000000000000000000000000000000000000000000003 (TEST ONLY), origin [5fd78eb3/270028'/0'/
0'/0']
xpub6EaVZPPwamQQKVnw8V5wJjXHJHC4hqX3ET8wLc9cWnCMzUgNZ2Gx4B7u9PQHUE9KrPgrRVZnt2oXakKwFeQ3WuWKekiN1TKGzUat5vPAfkT
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/keys/key-3.xpub
# @3 — tier 2 — 2-of-2 with hash B, after older(32768) relative
# seed entropy 0x0000000000000000000000000000000000000000000000000000000000000004 (TEST ONLY), origin [92c4c8b2/270028'/0'/
0'/0']
xpub6DkD3EFiJE6o9VL2wwc5x9aSkZG7GVQzEEYPdaJwCGUfYza5Rsr11xk25aG9J29cfHpyxEr6L39oH4mTmtJXVKK7KPVtVgT7HznajRzKepK
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/keys/key-4.xpub
# @4 — tier 2 — 2-of-2 with hash B, after older(32768) relative
# seed entropy 0x0000000000000000000000000000000000000000000000000000000000000005 (TEST ONLY), origin [086c047e/270028'/0'/
0'/0']
xpub6EEvgwBtdaoGsFWY6pNAuYMyWFcYrPivnW1zLM7myyvjhrSgVxwgmqHFjBuY54o7tJbacmtFvT3UGWta9PZXPtbf575rfBAvdHS6FDGxkC1
[exit 0]

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/keys/key-5.xpub
# @5 — tier 3 — sole key, after(1173520) absolute
# seed entropy 0x0000000000000000000000000000000000000000000000000000000000000006 (TEST ONLY), origin [cef8224c/270028'/0'/
0'/0']
xpub6E6vdTn7c2KFho1tuTeb9SPYiHxmFPsxSZgLzphMMDBu6LHZjDSX8Huf6sy9cZb3Bwwib3ZY92P1RPqPhcR4X7thfAzzhHELKrSJqdGm2EL
[exit 0]
```

The passphrases, and what the policy commits to

These are engraved as TEXT + QR plates rather than as `ms1`: `ms encode` takes a BIP-39 mnemonic or hex entropy, and these are human passphrases.

> **The hash is over the UTF-8 bytes with NO trailing newline.**
> `printf %s`, never `echo`. A stray `\n` changes the hash, and the tier it belongs to is then locked forever — with a plate beside it that looks correct.

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/preimages/preimage-0.txt
correct horse battery staple vault alpha
$ printf %s '<that file>' | sha256sum
ede0b28a805feca09a77c48f82babc1249c79aa840de94fa4a85bf55a453c813

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/preimages/preimage-1.txt
seven bridges over a quiet river bravo
$ printf %s '<that file>' | sha256sum
9ce09a8df1d1f8bc8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad

$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-hashvault/preimages/preimage-2.txt
the last plate rings twice charlie
$ printf %s '<that file>' | sha256sum
4743d7c47df21d29e3ed3dfec5d0c0a884ccc2708637dddf771c36d214056954
```

Tier 4 has no key, and the default parser refuses it

rust-miniscript answers `All spend paths must require a signature`. That is a **safety policy, not a language rule**: `and_v(v:after(N),sha256(H))` is well-formed miniscript that compiles to valid script. `sanity_check()` applies five rules and `requires_sig()` is the first; the library documents them as guarantees rather than validity — "results cannot be relied upon" — and ships `ExtParams` precisely so individual rules can be relaxed.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d54
7bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc1249c79aa840de94fa4a85bf55a453c813),multi_a(3,@0/270028'/0'/
0'/0'/<0;1>/*,@1/270028'/0'/0'/0'/<0;1>/*,@2/270028'/0'/0'/0'/<0;1>/*)),{and_v(v:older(32768),and_v(v:sha256(9ce09a8df1d1f8b
c8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,@3/270028'/0'/0'/0'/<0;1>/*,@4/270028'/0'/0'/0'/<0;1>/*))),{and
_v(v:after(1173520),pk(@5/270028'/0'/0'/0'/<0;1>/*)),and_v(v:after(1383520),sha256(4743d7c47df21d29e3ed3dfec5d0c0a884ccc2708
637dddf771c36d214056954))}}}) --group-size 0
md: template parse error: miniscript parse failed: All spend paths must require a signature
[exit 1]

--- and with the flag ---

$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d54
7bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc1249c79aa840de94fa4a85bf55a453c813),multi_a(3,@0/270028'/0'/
0'/0'/<0;1>/*,@1/270028'/0'/0'/0'/<0;1>/*,@2/270028'/0'/0'/0'/<0;1>/*)),{and_v(v:older(32768),and_v(v:sha256(9ce09a8df1d1f8b
c8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,@3/270028'/0'/0'/0'/<0;1>/*,@4/270028'/0'/0'/0'/<0;1>/*))),{and
_v(v:after(1173520),pk(@5/270028'/0'/0'/0'/<0;1>/*)),and_v(v:after(1383520),sha256(4743d7c47df21d29e3ed3dfec5d0c0a884ccc2708
637dddf771c36d214056954))}}}) --group-size 0 --experimental
chunk-set-id: 0x32d89
md1fxtvfpspp2040vepppppqqxqu2nxh0duzeg4qzlajsf5a7y37pt40qjf8re42zqmcu2f6fyh39ju2s
md1fxtvfps2205j59ha26g57gzvsyypg2nxhqqqzqqqnxhvuuzdgmuw3lz7g3yjyr2g6r2u6yrzhvgt3
md1fxtvfpshtxrg2rz78mwqmkran74xxmyryu7mk45szzuz5e4kqq3aqgz4fntvqp2q45nzqzs5e3s8r
md1fxtvfpslrqw5ws7hc37ly8ffu0knmlk96rq23pxvcfcgvd7amam3cdkjzszkj4qqldk6mrjrhcc87
warning: --experimental relaxed the signature rule. This descriptor has at least one spend path that needs NO key, so whoeve
r learns its preimage can spend it alone. If that preimage is engraved, THE PLATE IS BEARER ACCESS. Malleability, resource l
imits, repeated keys and timelock mixing were still checked.
note: stdout is a keyless descriptor template (no keys)
[exit 0]

The engraved template set is 4 md1 chunk(s).
```

`--experimental` relaxes ONLY that rule.

Malleability, resource limits, repeated keys and timelock mixing are still enforced per leaf — those admit scripts that are *unspendable* rather than merely unguaranteed, which is a worse class of mistake. And it warns on every use, because the card carries no record that a flag authored it.

What tier 4 means in practice.

After block 1,383,520, whoever holds passphrase C can spend this wallet alone. That passphrase is engraved on a plate. **The plate is bearer access.** That is the design, not an oversight — but it is the reason this wallet's plates are not all equal, and the reason the passphrase plates want a different hiding place from the key plates.

The cards

The keyless template — what gets engraved

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md verify --template tr(50929b74c1a04954b78b4b6035e97a5e078a5a
0f28ec96d547bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc1249c79aa840de94fa4a85bf55a453c813),multi_a(3,@0/27
0028'/0'/0'/<0;1>/*,@1/270028'/0'/0'/0'/<0;1>/*,@2/270028'/0'/0'/0'/<0;1>/*)),{and_v(v:older(32768),and_v(v:sha256(9ce09a
8df1d1f8bc8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,@3/270028'/0'/0'/0'/<0;1>/*,@4/270028'/0'/0'/0'/<0;1>/
*))),{and_v(v:after(1173520),pk(@5/270028'/0'/0'/0'/<0;1>/*)),and_v(v:after(1383520),sha256(4743d7c47df21d29e3ed3dfec5d0c0a8
84ccc2708637dddf771c36d214056954))}}}) md1fxtvfpspp2040vepppppqqxqu2nxh0duzeg4qzlajsf5a7y37pt40qjf8re42zqmcu2f6fyh39ju2s md1f
xtvfps2205j59ha26g57gzvsyypg2nxhqqqzqqqnxhvuuzdgmuw3lz7g3yjyr2g6r2u6yrzhvgt3 md1fxtvfpshtxrg2rz78mwqmkran74xxmyryu7mk45szzuz
5e4kqq3aqgz4fntvqp2q45nzqzs5e3s8r md1fxtvfpslrqw5ws7hc37ly8ffu0knmlk96rq23pxvcfcgvd7amam3cdkjzszkj4qqldk6mrjrhcc87 --experim
ental
OK
[exit 0]
```

What the cards carry

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fxtvfpspp2040vepppppqqxqu2nxh0duzeg4qzlajsf5a7y37
pt40qjf8re42zqmcu2f6fyh39ju2s md1fxtvfps2205j59ha26g57gzvsyypg2nxhqqqzqqqnxhvuuzdgmuw3lz7g3yjyr2g6r2u6yrzhvgt3 md1fxtvfpshtx
rg2rz78mwqmkran74xxmyryu7mk45szzuz5e4kqq3aqgz4fntvqp2q45nzqzs5e3s8r md1fxtvfpslrqw5ws7hc37ly8ffu0knmlk96rq23pxvcfcgvd7amam3c
dkjzszkj4qqldk6mrjrhcc87
template: tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc
1249c79aa840de94fa4a85bf55a453c813),multi_a(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*)),{and_v(v:older(32768),and_v(v:sha256(9ce09a
8df1d1f8bc8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,@3/<0;1>/*,@4/<0;1>/*))),{and_v(v:after(1173520),pk(@
5/<0;1>/*)),and_v(v:after(1383520),sha256(4743d7c47df21d29e3ed3dfec5d0c0a884ccc2708637dddf771c36d214056954))}}})
n: 6
wallet-policy-mode: false
md1-encoding-id: 32d898ab04d9725fe084ffe360e1c2f4
wallet-descriptor-template-id: 68a1a888385797337ce5debc90fcfb1e
origins:
  @0: m/270028'/0'/0'/0'
  @1: m/270028'/0'/0'/0'
  @2: m/270028'/0'/0'/0'
  @3: m/270028'/0'/0'/0'
  @4: m/270028'/0'/0'/0'
  @5: m/270028'/0'/0'/0'
wallet-policy-id: f97fdce3d1d0096f9fe7f8bc82f5648b
wallet-policy-id-fingerprint: 0xf97fdce3
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

The keyed set, and the bind

Two card sets, one wallet. The **template id** is key-stable, so it is the same across the keyless and keyed forms — which is exactly what makes it the right thing to compare.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d54
7bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc1249c79aa840de94fa4a85bf55a453c813),multi_a(3,@0/270028'/0'/
0'/0'/<0;1>/*,@1/270028'/0'/0'/0'/<0;1>/*,@2/270028'/0'/0'/0'/<0;1>/*)),{and_v(v:older(32768),and_v(v:sha256(9ce09a8df1d1f8b
c8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,@3/270028'/0'/0'/0'/<0;1>/*,@4/270028'/0'/0'/0'/<0;1>/*))),{and
_v(v:after(1173520),pk(@5/270028'/0'/0'/0'/<0;1>/*)),and_v(v:after(1383520),sha256(4743d7c47df21d29e3ed3dfec5d0c0a884ccc2708
637dddf771c36d214056954))}}}) --key @0=xpub6E6MpT6SMBy1Poaj5rEJ4DN5E73e58TWta6SVLs4eYpJ6hDNKqhwLxFcvMUtJnDSQJpdkThXEma9t23Wi
HhrYAJbK9GUu672njHHFenKt88 --fingerprint @0=39ec1b6e --key @1=xpub6F5JvRbobuuiwNQLy9ifRCK1HbewqyYk3PkMPNsrgkPcGGuDYXdYUDmHqM
sQeBzq58tf8eRxLaTWJPaa1pcE9TmRZTAFVk7U61UxQtMUNFj --fingerprint @1=d02bcedf --key @2=xpub6EaVZPPwamQQKVnw8V5wJjXHJHC4hqX3ET8
wLc9cWnCMzUgNZ2Gx4B7u9PQHUE9KrPgrRVZnt2oXakKwFeQ3WuWKekiN1TKGzUat5vPAfkT --fingerprint @2=5fd78eb3 --key @3=xpub6DkD3EFiJE6o
9VL2wwc5x9aSkZG7GVQzEEYPdaJwCGUfYza5Rsr11xk25aG9J29cfHpyxEr6L39oH4mTmtJXVKK7KPVtVgT7HznajRzKepK --fingerprint @3=92c4c8b2 --
key @4=xpub6EEvgwBtdaoGsFWY6pNAuYMyWFcYrPivnW1zLM7myyvjhrSgVxwgmqHFjBuY54o7tJbacmtFvT3UGWta9PZXPtbf575rfBAvdHS6FDGxkC1 --fin
gerprint @4=086c047e --key @5=xpub6E6vdTn7c2KFho1tuTeb9SPYiHxmFPsxSZgLzphMMDBu6LHZjDSX8Huf6sy9cZb3Bwwib3ZY92P1RPqPhcR4X7thfA
zzhHELKrSJqdGm2EL --fingerprint @5=cef8224c --group-size 0 --experimental
chunk-set-id: 0x5c65a
md1ft3j68qpp2040vepppppqqxqu2nxh0duzeg4qzlajsf5a7y37pt40qjf8re42zqm6205jszxc4se5llmr2r
md1ft3j68q2zm74dy20ypxgzzq59fntsqqpqqqfntkwwpx5d78gl30ygjfzp484np59p30rsc4yezy26j9m5u
md1ft3j68qkmsxaslvl4f3keqe88ka4dyqshq4xddsqy0gzq42v6mqq23ccr4r5847y0heps9l67a05dt0knv
md1ft3j68qa98376007chgvp2yyenp8pp3hmh0hw8pk6g2q2625p35s88kpkm36q4uah6tlqwj2e9qdewahkh
md1ft3j68p9uwkde93xgk2qsmqy06ua7pzfstxyyz3zxsepgqyfnq77a5wjjm5algyuj9x3qqeznva37mzkkw
md1ft3j68p080xt2r2gurh8eg76rf7wquey879utkmgyfcq2cmwgr5jg8guvv8x0laek8pmsgxjpa2xpsrqcu
md1ft3j68p3z838gk0xtf6vgukk6k3n7fad605n0qh85kk36vwn925x83qqpfejlmdc9w27qlpf9ljlhdat7q
md1ft3j68paap6qmnmjamh02w3ea4gsaf2qthswkxx3pt86wlw35cqea62sr5mtsdu92539qspxlk86fzj20z
md1ft3j68zrnec8vvwp6m86xny4ee5emm35vyjr3n9dvqns59trd4nu57vppmaxrx72mlxrqrpzvp8p8vnfhh
md1ft3j68z2gtf5myx93n9v8m2rtwgsuv8v4pczrx3s2psc4x342fmln4pcwk7dsf3rnj5ysn433nmdq42vn0
md1ft3j68zk7y2x8ztx8rvt8u7fn6fnj7yg5zsg0yprhph3t5rm53lqrkh2ag0a5v5amlkusp0ma52gt4ydt2
md1ft3j68zlw4pqqsjnurdez74zz0x652t6f0ujaz62x47krxplzf763rxwsnz7zz4jgeu4qr5u2m8jyplgqh
```

```
md1ft3j68r8y233tf3e0s4scpv2j0nr3nq5xd05hc3nxxf2zmc50n5vjsftzlf7wqklx2uws5mkv4cfranke6
md1ft3j68r0gyv66k5fuy7qvphvvqfpm8w9nfgsz9c7swxhxvqfexn0ktr088l8cvw4sz2qsuscfxqtufkzzj
md1ft3j68rs69vpkder6z9x6g90uzs2fqg5m0nu7lhhpcpxh5x468n3jeqms6qv872fxvmz2j
warning: --experimental relaxed the signature rule. This descriptor has at least one spend path that needs NO key, so whoeve
r learns its preimage can spend it alone. If that preimage is engraved, THE PLATE IS BEARER ACCESS. Malleability, resource l
imits, repeated keys and timelock mixing were still checked.
note: stdout is watch-only — public keys only, cannot spend
[exit 0]

The keyed set is 15 md1 chunk(s).

keyless template-id: 68a1a888385797337ce5debc90fcfb1e
keyed   template-id: 68a1a888385797337ce5debc90fcfb1e
SAME WALLET.
```

Addresses, derived from the card

From the cards, not from the policy string — this proves what the cards carry, not what was typed to make them. A taproot address commits to the **taptree shape** as well as the keys, so a wrong tree gives wrong addresses even when every key is right.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md address md1ft3j68qpp2040vepppqqxqu2nxh0duzeg4qzlajsf5a7y37
pt40qjf8re42zqm6205jszxc4se5llmr2r md1ft3j68q2zm74dy20ypxgzzq59fntsqqpqqqfntkwwpx5d78gl30ygjfzp484np59p30rsc4yezy26j9m5u md1
ft3j68qkmsxaslvl4f3keqe88ka4dyqshq4xddsqy0gzq42v6mqq23ccr4r5847y0heps9l67a05dt0knv md1ft3j68qa98376007chgvp2yyenp8pp3hmh0hw8
pk6g2q2625p35s88kpkm36q4uah6tlqwj2e9qdewahkh md1ft3j68p9uwkde93xgk2qsmqy06ua7pzfstxyyz3zxsepgqyfnq77a5wjjm5algyuj9x3qqeznva3
7mzkkw md1ft3j68p080xt2r2gurh8eg76rf7wquey879utkmgyfcq2cmwgr5jg8guvv8x0laek8pmsgxjpa2xpsrqcu md1ft3j68p3z838gk0xtf6vgukk6k3n
7fad605n0qh85kk36vwn925x83qqpfejlmdc9w27qlpf9ljlhdat7q md1ft3j68paap6qmnmjamh02w3ea4gsaf2qthswkxx3pt86wlw35cqea62sr5mtsdu925
39qspxlk86fzj20z md1ft3j68zrnec8vvwp6m86xny4ee5emm35vyjr3n9dvqns59trd4nu57vppmaxrx72mlxrqrpzvp8p8vnfhh md1ft3j68z2gtf5myx93n
9v8m2rtwgsuv8v4pczrx3s2psc4x342fmln4pcwk7dsf3rnj5ysn433nmdq42vn0 md1ft3j68zk7y2x8ztx8rvt8u7fn6fnj7yg5zsg0yprhph3t5rm53lqrkh2
ag0a5v5amlkusp0ma52gt4ydt2 md1ft3j68zlw4pqqsjnurdez74zz0x652t6f0ujaz62x47krxplzf763rxwsnz7zz4jgeu4qr5u2m8jyplgqh md1ft3j68r8
y233tf3e0s4scpv2j0nr3nq5xd05hc3nxxf2zmc50n5vjsftzlf7wqklx2uws5mkv4cfranke6 md1ft3j68r0gyv66k5fuy7qvphvvqfpm8w9nfgsz9c7swxhxv
qfexn0ktr088l8cvw4sz2qsuscfxqtufkzzj md1ft3j68rs69vpkder6z9x6g90uzs2fqg5m0nu7lhhpcpxh5x468n3jeqms6qv872fxvmz2j --chain 0 --c
ount 2
bc1puvyd9zxz6uvz0y0ehq7r5qz6h30txl4mgr8fxl3dqjp6xzpsy0qsgpgyny
bc1ptnvw76k6kqcdn7ra9906xp0j5966wgjwjtrfjjd8u08m4r6rf8fsjkcc6w
note: stdout is watch-only — public keys only, cannot spend
[exit 0]

$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md address md1ft3j68qpp2040vepppqqxqu2nxh0duzeg4qzlajsf5a7y37
pt40qjf8re42zqm6205jszxc4se5llmr2r md1ft3j68q2zm74dy20ypxgzzq59fntsqqpqqqfntkwwpx5d78gl30ygjfzp484np59p30rsc4yezy26j9m5u md1
ft3j68qkmsxaslvl4f3keqe88ka4dyqshq4xddsqy0gzq42v6mqq23ccr4r5847y0heps9l67a05dt0knv md1ft3j68qa98376007chgvp2yyenp8pp3hmh0hw8
pk6g2q2625p35s88kpkm36q4uah6tlqwj2e9qdewahkh md1ft3j68p9uwkde93xgk2qsmqy06ua7pzfstxyyz3zxsepgqyfnq77a5wjjm5algyuj9x3qqeznva3
7mzkkw md1ft3j68p080xt2r2gurh8eg76rf7wquey879utkmgyfcq2cmwgr5jg8guvv8x0laek8pmsgxjpa2xpsrqcu md1ft3j68p3z838gk0xtf6vgukk6k3n
7fad605n0qh85kk36vwn925x83qqpfejlmdc9w27qlpf9ljlhdat7q md1ft3j68paap6qmnmjamh02w3ea4gsaf2qthswkxx3pt86wlw35cqea62sr5mtsdu925
39qspxlk86fzj20z md1ft3j68zrnec8vvwp6m86xny4ee5emm35vyjr3n9dvqns59trd4nu57vppmaxrx72mlxrqrpzvp8p8vnfhh md1ft3j68z2gtf5myx93n
9v8m2rtwgsuv8v4pczrx3s2psc4x342fmln4pcwk7dsf3rnj5ysn433nmdq42vn0 md1ft3j68zk7y2x8ztx8rvt8u7fn6fnj7yg5zsg0yprhph3t5rm53lqrkh2
ag0a5v5amlkusp0ma52gt4ydt2 md1ft3j68zlw4pqqsjnurdez74zz0x652t6f0ujaz62x47krxplzf763rxwsnz7zz4jgeu4qr5u2m8jyplgqh md1ft3j68r8
y233tf3e0s4scpv2j0nr3nq5xd05hc3nxxf2zmc50n5vjsftzlf7wqklx2uws5mkv4cfranke6 md1ft3j68r0gyv66k5fuy7qvphvvqfpm8w9nfgsz9c7swxhxv
qfexn0ktr088l8cvw4sz2qsuscfxqtufkzzj md1ft3j68rs69vpkder6z9x6g90uzs2fqg5m0nu7lhhpcpxh5x468n3jeqms6qv872fxvmz2j --chain 1 --c
ount 2
bc1p3dgh5j4etfscn5nmmuqhnju58lhl2jw0snnnuf4hqtdyw3uldpcqercf6r
bc1pg2ufelj9t7jvvexdmlxjekyymjr6evl7wm8t2uzakpqs8z5yugjs4kcuc2
note: stdout is watch-only — public keys only, cannot spend
[exit 0]
```

chain	index	address
receive	0	bc1puvyd9zxz6uvz0y0ehq7r5qz6h30txl4mgr8fxl3dqjp6xzpsy0qsgpgyny
receive	1	bc1ptnvw76k6kqcdn7ra9906xp0j5966wgjwjtrfjjd8u08m4r6rf8fsjkcc6w
change	0	bc1p3dgh5j4etfscn5nmmuqhnju58lhl2jw0snnnuf4hqtdyw3uldpcqercf6r
change	1	bc1pg2ufelj9t7jvvexdmlxjekyymjr6evl7wm8t2uzakpqs8z5yugjs4kcuc2

The concrete descriptor

The whole wallet in one string: every key, every hash, every timelock. This is what a coordinator wants pasted in.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md descriptor md1ft3j68qpp2040vepppqqxqu2nxh0duzeg4qzlajsf5a7
y37pt40qjf8re42zqm6205jszxc4se5llmr2r md1ft3j68q2zm74dy20ypxgzzq59fntsqqpqqqfntkwwpx5d78gl30ygjfzp484np59p30rsc4yezy26j9m5u
md1ft3j68qkmsxaslvl4f3keqe88ka4dyqshq4xddsqy0gzq42v6mqq23ccr4r5847y0heps9l67a05dt0knv md1ft3j68qa98376007chgvp2yyenp8pp3hmh0
hw8pk6g2q2625p35s88kpkm36q4uah6tlqwj2e9qdewahkh md1ft3j68p9uwkde93xgk2qsmqy06ua7pzfstxyyz3zxsepgqyfnq77a5wjjm5algyuj9x3qqezn
va37mzkkw md1ft3j68p080xt2r2gurh8eg76rf7wquey879utkmgyfcq2cmwgr5jg8guvv8x0laek8pmsgxjpa2xpsrqcu md1ft3j68p3z838gk0xtf6vgukk6
k3n7fad605n0qh85kk36vwn925x83qqpfejlmdc9w27qlpf9ljlhdat7q md1ft3j68paap6qmnmjamh02w3ea4gsaf2qthswkxx3pt86wlw35cqea62sr5mtsdu
92539qspxlk86fzj20z md1ft3j68zrnec8vvwp6m86xny4ee5emm35vyjr3n9dvqns59trd4nu57vppmaxrx72mlxrqrpzvp8p8vnfhh md1ft3j68z2gtf5myx
93n9v8m2rtwgsuv8v4pczrx3s2psc4x342fmln4pcwk7dsf3rnj5ysn433nmdq42vn0 md1ft3j68zk7y2x8ztx8rvt8u7fn6fnj7yg5zsg0yprhph3t5rm53lqr
kh2ag0a5v5amlkusp0ma52gt4ydt2 md1ft3j68zlw4pqqsjnurdez74zz0x652t6f0ujaz62x47krxplzf763rxwsnz7zz4jgeu4qr5u2m8jyplgqh md1ft3j6
8r8y233tf3e0s4scpv2j0nr3nq5xd05hc3nxxf2zmc50n5vjsftzlf7wqklx2uws5mkv4cfranke6 md1ft3j68r0gyv66k5fuy7qvphvvqfpm8w9nfgsz9c7swx
hxvqfexn0ktr088l8cvw4sz2qsuscfxqtufkzzj md1ft3j68rs69vpkder6z9x6g90uzs2fqg5m0nu7lhhpcpxh5x468n3jeqms6qv872fxvmz2j
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc1249c79aa8
40de94fa4a85bf55a453c813),multi_a(3,[39ec1b6e/270028'/0'/0'/0']xpub661MyMwAqRbcFMFJZ12DzU5adoUivgK5xCwDYMxC5UtTBRgwLXEjCzEDX
RWWZvPxWmecGx1wCjNsiPhXcuaruVrdBLsH6YUzhyru8gcSM8x/<0;1>/*,[d02bcedf/270028'/0'/0'/0']xpub661MyMwAqRbcEq1guB2Dk15LELDLjDKnPY
gNC4gRtjPrEhJVroxpQheWf6WKUggvm53V8Q6EsHWwbscTAiEfESspxWDqJG9iVLQ96wt1FMr/<0;1>/*,[5fd78eb3/270028'/0'/0'/0']xpub661MyMwAqRb
cGEkmscab1t5uovpwez4gX8LECkkeuJgpjbmbj8Hgs5ceurpkWWgTptyWDfDhLV3YgSiXbTd5NhyFp6mwCX3xYYVXkBxpzk2/<0;1>/*)),{and_v(v:older(32
768),and_v(v:sha256(9ce09a8df1d1f8bc8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,[92c4c8b2/270028'/0'/0'/0']x
pub661MyMwAqRbcFPc8eXw9DQ7rqUScMmx4N9PcaqvwemivT9mTEWSJeoJyF8K8B5PLsvPwzgJeQ8DWHWmv52AfJHTQqpnTx2TdhoAWpiU1nx2/<0;1>/*,[086c
047e/270028'/0'/0'/0']xpub661MyMwAqRbcGRf7TNwDwRF8SoRHvZEU7xj4eYUkYG1d4AQc2pxkJmZurzmYaaJ8wgtfrmQbVtRY3XreZFsdTsU9PRQ4Q5Qj17
cp4DmMY2G/<0;1>/*))),{and_v(v:after(1173520),pk([cef8224c/270028'/0'/0'/0']xpub661MyMwAqRbcFbjUAobTvfu9fkXEZS8VoB5jMdcdQP2DJ
wHjkyNmnxkhquwzerhugnad1WHQT4dn7GPqPJXA6v5t2vPxJjoswhxQ6KEPLd4/<0;1>/*)),and_v(v:after(1383520),sha256(4743d7c47df21d29e3ed3
dfec5d0c0a884ccc2708637dddf771c36d214056954))}}})#he4degaz
note: stdout is watch-only — public keys only, cannot spend
[exit 0]

That descriptor is 1299 bytes.
```

```
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc1249c79aa8
40de94fa4a85bf55a453c813),multi_a(3,[39ec1b6e/270028'/0'/0'/0']xpub661MyMwAqRbcFMFJZ12DzU5adoUivgK5xCwDYMxC5UtTBRgwLXEjCzEDX
RWWZvPxWmecGx1wCjNsiPhXcuaruVrdBLsH6YUzhyru8gcSM8x/<0;1>/*,[d02bcedf/270028'/0'/0'/0']xpub661MyMwAqRbcEq1guB2Dk15LELDLjDKnPY
gNC4gRtjPrEhJVroxpQheWf6WKUggvm53V8Q6EsHWwbscTAiEfESspxWDqJG9iVLQ96wt1FMr/<0;1>/*,[5fd78eb3/270028'/0'/0'/0']xpub661MyMwAqRb
cGEkmscab1t5uovpwez4gX8LECkkeuJgpjbmbj8Hgs5ceurpkWWgTptyWDfDhLV3YgSiXbTd5NhyFp6mwCX3xYYVXkBxpzk2/<0;1>/*)),{and_v(v:older(32
768),and_v(v:sha256(9ce09a8df1d1f8bc8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,[92c4c8b2/270028'/0'/0'/0']x
pub661MyMwAqRbcFPc8eXw9DQ7rqUScMmx4N9PcaqvwemivT9mTEWSJeoJyF8K8B5PLsvPwzgJeQ8DWHWmv52AfJHTQqpnTx2TdhoAWpiU1nx2/<0;1>/*,[086c
047e/270028'/0'/0'/0']xpub661MyM…
```

The six key cards

An `mk1` card binds a cosigner's key to the wallet. They bind to the **keyless template**, not the keyed set: the stub is form-aware, so a card minted against the keyed policy carries a different stub and is refused against the template. **One wallet, two stubs.**

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/keys.txt
[39ec1b6e/270028'/0'/0'/0']xpub6E6MpT6SMBy1Poaj5rEJ4DN5E73e58TWta6SVLs4eYpJ6hDNKqhwLxFcvMUtJnDSQJpdkThXEma9t23WiHhrYAJbK9GUu
672njHHFenKt88
[d02bcedf/270028'/0'/0'/0']xpub6F5JvRbobuuiwNQLy9ifRCK1HbewqyYk3PkMPNsrgkPcGGuDYXdYUDmHqMsQeBzq58tf8eRxLaTWJPaa1pcE9TmRZTAFV
k7U61UxQtMUNFj
[5fd78eb3/270028'/0'/0'/0']xpub6EaVZPPwamQQKVnw8V5wJjXHJHC4hqX3ET8wLc9cWnCMzUgNZ2Gx4B7u9PQHUE9KrPgrRVZnt2oXakKwFeQ3WuWKekiN1
TKGzUat5vPAfkT
[92c4c8b2/270028'/0'/0'/0']xpub6DkD3EFiJE6o9VL2wwc5x9aSkZG7GVQzEEYPdaJwCGUfYza5Rsr11xk25aG9J29cfHpyxEr6L39oH4mTmtJXVKK7KPVtV
gT7HznajRzKepK
[086c047e/270028'/0'/0'/0']xpub6EEvgwBtdaoGsFWY6pNAuYMyWFcYrPivnW1zLM7myyvjhrSgVxwgmqHFjBuY54o7tJbacmtFvT3UGWta9PZXPtbf575rf
BAvdHS6FDGxkC1
[cef8224c/270028'/0'/0'/0']xpub6E6vdTn7c2KFho1tuTeb9SPYiHxmFPsxSZgLzphMMDBu6LHZjDSX8Huf6sy9cZb3Bwwib3ZY92P1RPqPhcR4X7thfAzzh
HELKrSJqdGm2EL
[exit 0]

note: stdout is watch-only — public keys only, cannot spend
$ /scratch/code/shibboleth/mnemonic-engrave/design/journeys/mk_cards_from_json.py /scratch/code/shibboleth/mnemonic-engrave/
design/journeys/out/hashvault/mk-encode.json /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/mk-enco
de-raw.txt
6 key cards -> 18 mk1 chunks
[exit 0]

template-id prefix: 68a1a888   stub mk embedded: 68a1a888
```

What goes on which plate

The device caps a QR at `dim 37` — QR v5, about 106 bytes at ECC-L (`engrave/engrave.go:420`). Measured against that cap, the three kinds of content land in three different places:

```
  device QR cap (dim 37 = QR v5, ECC-L byte mode): ~106 bytes
  single-plate TEXT budget:                        300 chars

  md1 chunks   4, longest  80 chars  -> TEXT + QR    (fits)
  mk1 chunks  18, longest 111 chars  -> TEXT ONLY    (over the QR cap)
  descriptor      1298 bytes          -> 5 TEXT plates (12.2x the QR cap)

The mk1 cards lose their QR to 5 characters: 111 against a 106-byte cap.
Nothing is lost by it -- an mk1 is BCH-protected either way -- but it is
the difference between scanning a plate and typing it.

$ /scratch/code/shibboleth/mnemonic-engrave/design/journeys/split_descriptor_plates.py /scratch/code/shibboleth/mnemonic-eng
rave/design/journeys/out/hashvault/descriptor.txt /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/de
scriptor-plates
descriptor: 1298 bytes -> 5 TEXT plates (300-char budget, 16 for the header)
sha256:     2c5a381fbb1797176ac4f10759b3aef8e55d650cccfccf0e2896085a69404572
  NOTE: these plates carry NO checksum. md1 does; this does not.
  part 1/5  284 chars
  part 2/5  284 chars
  part 3/5  284 chars
  part 4/5  284 chars
  part 5/5  162 chars
  rejoin check: 5 parts -> 1298 bytes, sha256 matches
[exit 0]

The five descriptor plates, as they will read:

$ cat descriptor-part-1-of-5.txt
part 1/5
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82b ...

$ cat descriptor-part-2-of-5.txt
part 2/5
YUzhyru8gcSM8x/<0;1>/*,[d02bcedf/270028'/0'/0'/0']xpub661MyMwAqRbcEq1guB2Dk15LELDLjDKnPYgNC4gRtjPrEhJVroxpQheWf ...

$ cat descriptor-part-3-of-5.txt
part 3/5
5NhyFp6mwCX3xYYVXkBxpzk2/<0;1>/*)),{and_v(v:older(32768),and_v(v:sha256(9ce09a8df1d1f8bc8892441a9eb30d0a18bc7db ...

$ cat descriptor-part-4-of-5.txt
part 4/5
x2/<0;1>/*,[086c047e/270028'/0'/0'/0']xpub661MyMwAqRbcGRf7TNwDwRF8SoRHvZEU7xj4eYUkYG1d4AQc2pxkJmZurzmYaaJ8wgtfr ...
… clipped; full text in transcript_hashvault.txt
```

The mk1 cards lose their QR to five characters

— 111 against a 106-byte cap. Nothing is lost by it; an mk1 is BCH-protected either way. It is the difference between scanning a plate and typing one.

The descriptor plates carry NO checksum.

An md1 chunk has a BCH checksum and a shared chunk-set-id, so a mis-transcribed character is detected and a mixed-up set is refused. These five plates are raw text split at a byte offset: one wrong character yields a descriptor that is still syntactically plausible and describes a *different wallet*. They are a convenience for a coordinator. **The md1 card set is the backup.**

The bundle, and the secrets

```
The engraved set: 4 md1 + 18 mk1 = 22 public strings.
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me bundle --in /scratch/code/shibboleth/mnemonic-engrave/design/j
ourneys/out/hashvault/backup-strings.txt --preview /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/p
lates --png --manifest /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/manifest.json
me: rendered plate 1 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-1.png
me: rendered plate 2 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-2.png
me: rendered plate 3 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-3.png
me: rendered plate 4 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-4.png
me: rendered plate 5 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-5.png
me: rendered plate 6 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-6.png
me: rendered plate 7 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-7.png
me: rendered plate 8 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-8.png
me: rendered plate 9 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-9.png
me: rendered plate 10 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-10.png
me: rendered plate 11 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-11.png
me: rendered plate 12 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-12.png
me: rendered plate 13 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-13.png
me: rendered plate 14 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-14.png
me: rendered plate 15 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-15.png
me: rendered plate 16 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-16.png
me: rendered plate 17 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-17.png
me: rendered plate 18 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-18.png
me: rendered plate 19 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-19.png
me: rendered plate 20 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-20.png
me: rendered plate 21 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-21.png
me: rendered plate 22 → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/plates/plate-22.png
me: wrote manifest to /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/hashvault/manifest.json
me: backup needs 23 plates (22 public + ms1 on device):
  plate 1/23  md1 policy  → push via NFC & engrave
  plate 2/23  md1 policy  → push via NFC & engrave
  plate 3/23  md1 policy  → push via NFC & engrave
  plate 4/23  md1 policy  → push via NFC & engrave
… clipped; full text in transcript_hashvault.txt
```

The secrets

`me` refuses an `ms1` outright — it will not render, preview or transmit a secret, which is what makes the rest of the bundle safe to preview at all.

```
$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms encode --phrase abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon aban
don abandon diesel
ms10e ntrsq qqqqq qqqqq qqqqq qqqqq qqqqq qqqqq qqqqq qqqqq qqqqq qqqqq qzr3h cpkas 7yqr8
word count: 24
language: english (BIP-39 checksum valid)
passphrase: not stored in ms1 (record separately if used)
warning: stdout carries private key material (can spend) — redirect or encrypt (e.g. '> file.txt' or '| age -e ...')
[exit 0]

$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --in /dev/fd/63 --hex
me: refusing to emit ms1 over NFC: ms1 is secret seed entropy and must never be transmitted by radio (RF eavesdropping ris
k). Enter it by hand on the device: New > Input Seed > CODEX32.
[exit 3]
```

The gap: the engraved set does not name the slots

Every key sits at the **same path** from a **different seed**. So the keyless template's six origins are identical, and it carries no fingerprints — while the keyed card does. The engraved backup is the template plus the six mk1 cards, and those bind to the same template with the same stub at the same origin, so they add no signal either.

Slot order is load-bearing. `multi_a` commits to key *order*, so a different assignment is a different script, a different taptree leaf, and a different address:

```
The keyless template's origins, as engraved:
origins:
  @0: m/270028'/0'/0'/0'
  @1: m/270028'/0'/0'/0'
  @2: m/270028'/0'/0'/0'
  @3: m/270028'/0'/0'/0'
  @4: m/270028'/0'/0'/0'
  @5: m/270028'/0'/0'/0'
wallet-policy-id: f97fdce3d1d0096f9fe7f8bc82f5648b

The keyed card's origins, which DO name a master:
origins:
  @0: [39ec1b6e/270028'/0'/0'/0']
  @1: [d02bcedf/270028'/0'/0'/0']
  @2: [5fd78eb3/270028'/0'/0'/0']
  @3: [92c4c8b2/270028'/0'/0'/0']
  @4: [086c047e/270028'/0'/0'/0']
  @5: [cef8224c/270028'/0'/0'/0']
wallet-policy-id: a0b128ceaef3155a40af6f8e88765ecb

What a wrong assignment costs, measured:
  canonical  @0..@5 = key-0..key-5 : bc1puvyd9zxz6uvz0y0ehq7r5qz6h30txl4mgr8fxl3dqjp6xzpsy0qsgpgyny
  swap @0/@1 (tier 1 3-of-3)       : bc1p57pwugukl4a87nydmgymxmd779qv3qfyqu20elc60fyt5dhq7nfsststjq
  swap @3/@4 (tier 2 2-of-2)       : bc1p855ry9mfe2xxx509p92v2pgkgta53qkhufdty7gq6dguakcax9jq57dvx0

Three different wallets from one set of six keys. 6! = 720 assignments,
and the engraved template distinguishes none of them.
```

Three different wallets from one set of six keys, and the engraved template distinguishes none of them.

A restorer holding these plates and all six seeds has 720 candidate assignments and no way to choose. This is not a tooling defect — it is a property of the wallet's shape, and it is the thing this journey exists to have found before anything was cut.

The ways out, and the one that costs almost nothing

- **Declare the fingerprints in the template** — `md encode --fingerprint @0=… --fingerprint @1=…`, which is repeatable and was simply not passed. The six masters already have six distinct fingerprints. No path moves, no key is revealed, the policy is untouched, and the template id does not change. **This is the fix, and the next two pages measure it.**
- **Give each key its own account index** — `m/270028'/0'/N'/0'`, so the origins themselves differ. Works, but it changes every address and every derivation, which is a large price for something a declaration solves.
- **Engrave the KEYED card** instead of the template — a fingerprint per slot, at the cost of 15 chunks rather than 4 and of putting the cosigner xpubs on metal.
- **Record the assignment out of band** — the weakest, because it is then the one part of the backup that is neither engraved nor checkable.

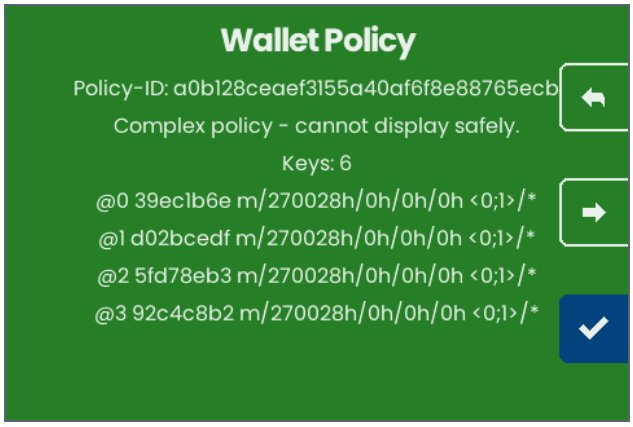
This is F-216 (mapping an mk1 to an `@N` slot) in its hardest form: there, seating matched on a declared origin because the origins differed. **And the sibling taproot journey is not safe from it either — checked, not assumed.** Its keyless template reads `@0`, `@4` and `@8` as the same `m/48'/0'/0'/2'`, one per master, with no fingerprints. The collision here is six-way and obvious; there it is three-way and hides inside a wallet that looks fine. Filed.

The device walk — two arms, and the second one is the point

Everything so far is host-side. This is the same card set carried into the SeedHammer II emulator over NFC by `capture_hashvault.py`, which reuses the seating walk's driver. Both arms **assert**: the screenshots below are comparisons that held, not photographs of whatever happened.

Arm 1 — the control: the keyed card, 15 chunks, no seating

Before the interesting arm can mean anything, the device has to be able to handle this wallet at all. Four tiers, two `sha256` hashlocks, both timelock flavours, `multi_a` at 3/2/1 and a NUMS internal key — a device that simply choked on that shape would *also* produce a refusal in arm 2, for an entirely different reason.



Arm 1: the consent screen, page 1 of 3 — the keyed card's Policy-ID.

Arm 1, page 2 — the derived addresses.

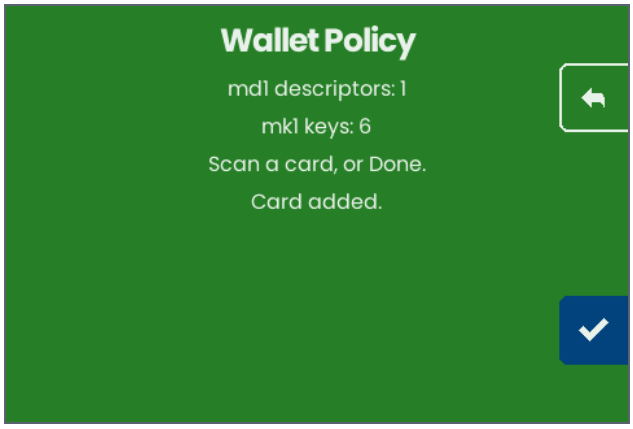
It derives. Read off the screen and compared against the host, which computed them in Rust from the same six xpubs:

matched on the device AND on the host
wallet id `a0b128ceaef3155a40af6f8e88765ecb`
`bc1puvyd9zxz6uvz0y0ehq7r5qz6h30txl4mgr8fxl3dqjp6xzpsy0qsgpgyny`
`bc1ptnvw76k6kqcdn7ra9906xp0j5966wgjwjtrfjjd8u08m4r6rf8fsjkcc6w`
`bc1p3dgh5j4etfscn5nmmuqhnju58lhl2jw0snnnuf4hqtdyw3uldpcqercf6r`
`bc1pg2ufelj9t7jvvexdmlxjekyymjr6evl7wm8t2uzakpqs8z5yugjs4kcuc2`

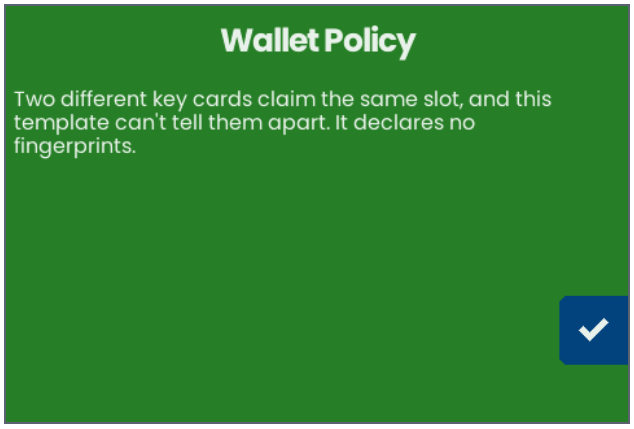
> Two implementations, two languages, one air gap, same answers. That is what makes arm 2's refusal attributable to the seating and nothing else.

Arm 2 — the engraved set, and the device declines to guess

Now the plates as they would actually be cut: the 4-chunk keyless template and all 6 mk1 key cards. The device gathers every one of them — the tally on the left confirms it — and then refuses.



Arm 2: 4 template chunks assembled, 6 mk1 key cards gathered. Nothing has been refused yet.



Arm 2: the refusal. No address anywhere on the screen.

The sentence is `gui/wallet_policy.go`'s, and it names the cause rather than merely reporting failure:

> “Two different key cards claim the same slot, and this template can't tell them apart. It declares no fingerprints.”

That is `errSeatSlotContested`. All six cards match all six slots on origin, the template declares no fingerprint to break the tie, and `seatKeyCards` treats every undecidable state as a refusal — because a misassignment does not fail loudly, it derives a *different wallet's* address and shows it to the operator as proof. **No address appears on this screen, and the walk asserts that too.**

The refusal check is not vacuous

An assertion that a string *appears* passes for a walk that quietly stopped driving the device. So `--prove-it-can-fail` feeds the keyed card — which derives, per arm 1 — while still demanding the refusal, and the run counts as passing only if the walk *fails*:

```
$ python3 capture_hashvault.py --prove-it-can-fail --no-build
NEGATIVE CONTROL: feeding the KEYED card while demanding the seating refusal.
  15 chunks, no key cards, so 'declares no fingerprints' must NOT appear.

NEGATIVE CONTROL PASSED: the walk refused to report a refusal that never happened.
$ echo $?
0
```

> The firmware anticipated this exact case before this wallet existed: the comment above that refusal reads “It happens when the template declares no fingerprints and two slots share a derivation path — a stripped template cannot tell them apart.” This journey is the first wallet to walk into it, and the device was already right.

The fix costs one chunk

`md encode` takes a repeatable `--fingerprint`. The template was encoded without it, so it declared bare paths. The six masters already have six distinct fingerprints — they are printed on the key cards — and declaring them is the whole repair:

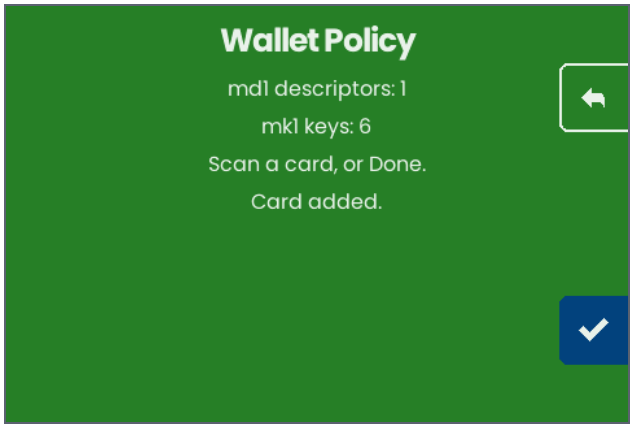
```
The gap is not that the paths collide -- it is that the template carries
NOTHING ELSE to tell the slots apart. `md encode` takes a repeatable
--fingerprint, and the six masters have six distinct fingerprints, so the
template can declare them without touching a path, a key or the policy.

$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d54
7bfee9ace803ac0,{and_v(v:sha256(ede0b28a805feca09a77c48f82babc1249c79aa840de94fa4a85bf55a453c813),multi_a(3,@0/270028'/0'/
0'/0'/<0;1>/*,@1/270028'/0'/0'/0'/<0;1>/*,@2/270028'/0'/0'/0'/<0;1>/*)),{and_v(v:older(32768),and_v(v:sha256(9ce09a8df1d1f8b
c8892441a9eb30d0a18bc7db81bb0fb3f54c6d9064e7b76ad),multi_a(2,@3/270028'/0'/0'/0'/<0;1>/*,@4/270028'/0'/0'/0'/<0;1>/*))),{and
_v(v:after(1173520),pk(@5/270028'/0'/0'/0'/<0;1>/*)),and_v(v:after(1383520),sha256(4743d7c47df21d29e3ed3dfec5d0c0a884ccc2708
637dddf771c36d214056954))}}}) --group-size 0 --experimental --fingerprint @0=39ec1b6e --fingerprint @1=d02bcedf --fingerprin
t @2=5fd78eb3 --fingerprint @3=92c4c8b2 --fingerprint @4=086c047e --fingerprint @5=cef8224c
chunk-set-id: 0x00d82
md1fqrvzzqpp2040veppppqqxqu2nxh0duzeg4qzlajsf5a7y37pt40qjf8re42zqx6mqa80ahfvpj
md1fqrvzzqt0ff7j2skl4tfzneqfjqss9p2v6uqqqgqqzv6annsf4r0368utezyjgc2rty2pdjletd
md1fqrvzzqjp484np59p30rahqdmp7el2nrdjpjw0dm26gppwp2v6mqqg7syp25e5q7yl59skuwn57
md1fqrvzzq7cqz5wxqagapa03ra7gwjncld8hlvt5xq4zzvesnsscmamhmhrsmdyyrk6djz5tls2wy
md1fqrvzzpyq454grrfqw0vrdhr5ptem05h7h36ehykyezegzrvq3ltnhcyfxq5yrmdta2t8avv
warning: --experimental relaxed the signature rule. This descriptor has at least one spend path that needs NO key, so whoeve
r learns its preimage can spend it alone. If that preimage is engraved, THE PLATE IS BEARER ACCESS. Malleability, resource l
imits, repeated keys and timelock mixing were still checked.
note: stdout is a keyless descriptor template (no keys)
[exit 0]

Chunks: 4 without fingerprints, 5 with. The whole cost is 1.

It is the SAME WALLET -- the template id is key-stable and does not move:
  bare        68a1a888385797337ce5debc90fcfb1e
  fingerprint 68a1a888385797337ce5debc90fcfb1e

And the slots are now distinguishable:
  @0: [39ec1b6e/270028'/0'/0'/0']
  @1: [d02bcedf/270028'/0'/0'/0']
  @2: [5fd78eb3/270028'/0'/0'/0']
… clipped; full text in transcript_hashvault.txt
```

Four chunks became five. The paths did not move, no key is revealed, the policy is character-for-character the same, and the transcript gates on the wallet-descriptor-template-id being unchanged — `68a1a888…` before and after — so this is demonstrably the same wallet and not a new one that happens to work.

> Worth being precise about what was wrong: not the paths. Sharing `m/270028'/0'/0'/0'` across six masters is legitimate and common — it is the standard shape for multisig, where every cosigner uses the same account path. What was wrong is that the template carried **nothing else to tell the slots apart**, and it had a field for exactly that.

Arm 3 — the same six cards, and now the device seats them

Nothing about the cards changed. The only difference from arm 2 is which template they are presented against.



Arm 3: the 5-chunk fingerprinted template and the same 6 mk1 cards. No refusal this time.

Arm 3: the addresses, seated from the key cards rather than read off a keyed card.

derived on the device, matching the host
bc1puvyd9zxz6uvz0y0ehq7r5qz6h30txl4mgr8fxl3dqjp6xzpsy0qsgpgyny
bc1ptnvw76k6kqcdn7ra9906xp0j5966wgjwjtrfjjd8u08m4r6rf8fsjkcc6w
bc1p3dgh5j4etfscn5nmmuqhnju58lhl2jw0snnnuf4hqtdyw3uldpcqercf6r
bc1pg2ufelj9t7jvvexdmlxjekyymjr6evl7wm8t2uzakpqs8z5yugjs4kcuc2

The same four addresses the keyed card produced in arm 1, and the same four the host derived in Rust — but reached the way a restorer would actually reach them: from a keyless template and one card per key, with the device doing the join.

That is the round trip.
Six seeds went in; a template and six key cards came out; the device rebuilt the wallet from those and agreed with the host on every address. Arm 2 remains the more valuable half of the journey — it is the version that would have been engraved.

Three arms, three different things proved, and the order matters: arm 1 rules out the wallet's shape as an explanation, arm 2 shows the engraved set failing, arm 3 shows the repair working on the identical cards. Any one of them alone would have been ambiguous.

And the plates themselves cannot restore it

Everything up to here starts from the strings in `out/` — the generator's own output. An operator starts from a stack of plates. So the last step goes the other way: `restore_from_plates.py` decodes the QR out of each rendered plate image using `zbarimg` — an **independent** decoder, not the library that drew them — and rebuilds the wallet from what came off the images and nothing else.

```
Everything above starts from the strings in out/. An operator starts from
a stack of plates. restore_from_plates.py goes the other way: it decodes
the QR out of each rendered plate image with zbarimg -- an independent
decoder, not the library that drew them -- and rebuilds the wallet from
what came off the images and nothing else.

This journey's plates carry the BARE template, deliberately: they are the
set section 15 is about. So the restore is expected to FAIL, and the gate
below is inverted -- it goes red if these plates ever restore cleanly,
because that would mean the finding this journey documents has silently
stopped being true and the narrative around it is now wrong.

$ python3 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/restore_from_plates.py hashvault
FAIL: the engraved plates rebuild a template whose slots cannot be told apart -- one mk1 card would match several, and seati
ng must refuse
hashvault: 23 plates in the manifest

Layer 1 — the QR on each plate decodes to the string the manifest names:
  22 of 23 plates decoded byte-exact via zbarimg
  1 plate(s) carry NO rendered image and were NOT checked:
    plate 23 (ms1)

Rebuilding from the plate-borne strings alone: 4 md1 chunk(s), 18 mk1 string(s)

Layer 2 — the rebuilt wallet, against what the journey derived separately:
  template-id off the PLATES : 68a1a888385797337ce5debc90fcfb1e
  template-id from the run   : 68a1a888385797337ce5debc90fcfb1e

  slots 6, distinct declarations 1, with a fingerprint 0
  the plate-borne template is NOT SEATABLE
[exit 1]
… clipped; full text in transcript_hashvault.txt
```

The template-id off the plates is RIGHT.

The QR, the rendering, the codec and the chunking are all fine — 22 of 23 images decoded byte-exact. What the plates cannot do is say which key is which, and the seatability check is what catches it.

That is the finding stated one final way, and the most concrete one: not "this template is ambiguous" as a property of a data structure, but *these twenty-two pieces of metal do not add up to this wallet*.

The gate here is inverted, deliberately

This journey's plates carry the **bare** template on purpose — they are the set the gap pages are about. So the transcript requires the restore to **fail**, and goes red if these plates ever restore cleanly. If someone later fixes them without rewriting the narrative, that gate fires and says so. The sibling pathological journeys run the same driver expecting exit 0, with both of its controls.

The 23rd plate is an `ms1` entry with no rendered image, so it was not checked. Reported rather than skipped: a plate this driver did not look at is the difference between a partial result and a false clean one.

What this journey does NOT show

- **Still not metal.** The plate restore above reads a rendered PNG. The path from that PNG to scratched steel to a phone camera — legibility, depth, glare, engraving artefacts — is not exercised anywhere in this document. What is proved is the software chain end to end.
- **No REAL hardware.** The device walk is the emulator — the same Go firmware compiled to wasm, driven over the same NFC entry point, but not a machine with a screen and a hammer. Everything upstream of the display stack is the shipping code; the display stack itself is not exercised.
- **Only one seating attempt.** Arm 2 presents the six cards in one order. It refuses, so no other order was tried — and by the argument on the gap page, no order would have helped, because the refusal is about the cards being *indistinguishable*, not about their sequence.
- **No host-side restore test**, though the device now performs the restoring half: arm 3 rebuilds the wallet on a SeedHammer II from the fingerprinted template plus the six key cards and derives the right addresses. What is missing is the same walk done from the *engraved plates* — read back off metal rather than replayed from the files that produced them.
- **No hardware.** Nothing has been cut. The plate counts and the QR capacity are computed from the device's own constants, not measured on metal.
- **The descriptor plates are unchecked text.** Stated again because it is the one place this backup is weaker than the constellation's usual guarantees.

Reproducing this document

```
bash derive-hashvault-keys.sh
bash transcript_hashvault.sh > transcript_hashvault.txt
python3 build_pdf_hashvault.py
```

The builder refuses a transcript snapshot that does not match the script that produced it, so this document cannot quietly describe a toolchain that has moved on underneath it.